Design patterns and backend architecture

Chapter 7: Practice questions and model answers

Practice designs, not claims about an employer.

7. Practice questions and model answers

Question 1: "Explain SOLID without reciting the letters."

Answer: "SOLID helps me keep responsibilities clear, define reliable contracts, and control dependencies. For example, if I add a new payment route, I would not want to modify one giant class that handles validation, fees, storage, and provider calls. I would separate stable orchestration from the parts that vary. I use those principles when they make changes and tests safer, rather than creating abstractions everywhere."

Question 2: "Why would you choose Strategy over a switch statement?"

Answer: "If there are only two simple fixed cases, a switch can be clearer. I move to Strategy when policies grow, change independently, or need separate tests. Then each policy implements one contract and the caller does not contain every rule. I would also make the selection rule explicit so it does not become a hidden second switch."

Question 3: "How is Hexagonal different from layering?"

Answer: "Both can separate responsibilities. Hexagonal makes the dependency direction more explicit: business code defines the ports it needs, and database or API adapters implement them. That is useful when integrations change or business logic needs isolated tests. Layering alone does not guarantee that the business layer is free from infrastructure details."

Question 4: "When would you choose a modular monolith over microservices?"

Answer: "I would start with strong modules if the workflow is cohesive and one team can deploy it safely. A modular monolith keeps local transactions and debugging simpler. I would split services when independent ownership, release cadence, or scaling has enough value to justify network calls, data consistency, monitoring, and operational overhead."

Question 5: "Does the outbox guarantee exactly-once processing?"

Answer: "No. It protects the local database change and event record from a dual-write gap. The relay may publish a message more than once, and a consumer may retry. I would make the consumer idempotent using a stable event ID or business key and check the effect in durable storage."

Question 6: "Can you roll back a saga?"

Answer: "A saga uses compensating business actions after local transactions have committed. It is not a single database rollback. Some effects, such as an external payment already settled, may be irreversible; the right response could be a reversal or manual reconciliation. I would define that behavior before relying on the saga."

Question 7: "What if the provider call times out?"

Answer: "A timeout tells me our response did not arrive within the deadline; it does not tell me whether the provider acted. I would record an uncertain state and query or reconcile using a stable reference. I would retry only under the provider's idempotency contract and the business rule for duplicate prevention."

Question 8: "Which pattern did you use in your last project?"

Answer template: "The problem was [actual problem]. I used [pattern or design] by [specific implementation I owned]. I considered [alternative]. The tradeoff was [real cost], and I verified it with [test or production measure]." If a formal pattern name does not fit, describe the design without forcing a label.

Rapid oral drill

Give each answer in 30-45 seconds, then add one tradeoff: SRP versus tiny classes; DIP versus dependency injection; Strategy versus State; Adapter versus Facade; Repository versus generic CRUD wrapper; modular monolith versus microservices; command versus event; outbox versus saga; CQRS versus event sourcing; retry versus circuit breaker; timeout versus cancellation; bulkhead versus rate limit; cache invalidation; strangler migration rollback; one pattern you personally used.